Slices in Go

A dynamic, flexible view of an array that allows easy addition and removal of elements.

Why Slices?

Dynamic Data

Store collections that change size during program execution.

Flexible Size

Overcome fixed-size array limitations with easy resizing.

Efficient

Built-in functions make operations fast and intuitive.

Arrays vs Slices

Arrays

- Fixed size declared at creation
- Cannot change length
- Size is part of type
- Less flexible

Slices

- Dynamic size that can grow
- Easy to resize with built-ins
- More commonly used
- Flexible and powerful

Slice Syntax

Declaration

```
var s []int
```

Creates nil slice of integers.

Literal Syntax

```
s := []int{1, 2, 3}
```

Initialize with values.

Using make()

```
s := make([]int, 3)
```

Creates slice with length 3.

Creating Slices

Three Common Methods

1. **Literal syntax:** `[]int{1, 2, 3}`
2. **make() function:** `make([]int, 3)`
3. **From array:** `arr[1:3]`

The `[]int` notation represents a slice of integers.

Adding Elements

Initial Slice

Create slice with existing elements.

append() Function

Add new elements to the end.

Updated Slice

Get new slice with added elements.

Complete Example

```go
package main

import "fmt"

func main() {
    s := []int{10, 20, 30}
    s = append(s, 40)
    fmt.Println(s)
}
```

This program creates a slice with three elements, adds a fourth element using `append()`, and prints the result.

Program Output

[10 20 30 40]

Result

Final slice after appending 40

The slice grows dynamically from 3 to 4 elements. Index starts from 0, so `s[0]` is 10 and `s[3]` is 40.

Key Takeaways

Dynamic Growth

Slices can grow and shrink as needed during program execution.

Built-in Functions

Use `append()` to add elements and `make()` to create slices.

Common Practice

Slices are more commonly used than arrays in real Go applications.

Mastering Slices

Next Steps

- Practice creating and modifying slices
- Explore slicing syntax `slice[low:high]`
- Learn about capacity and length
- Understand slice internals